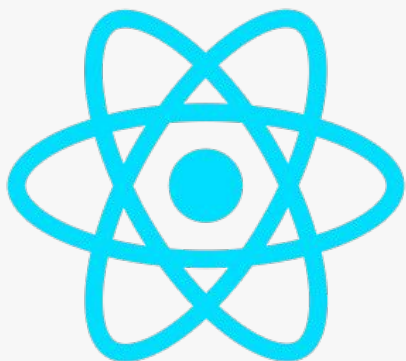




The Ultimate React Course



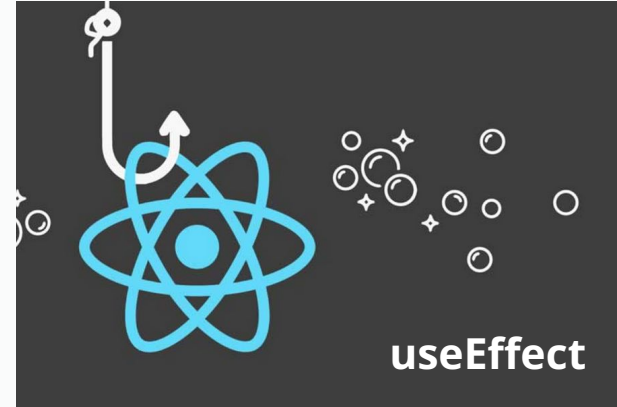
 @newtonschool

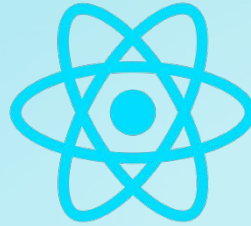
Lecture 17: UseEffect -2



Table of Contents

- Quick Recap
- Introduction to useEffect
- Side Effects in React (data fetching)
- Fetching data with fetch
- Lifting State Up
- Managing API using useState and useEffect
- Error Handling and Loading States
- Example: Displaying Public Data from a Public API





Quick Recap

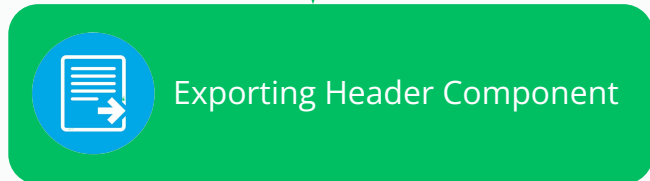
A brief Overview

Recap: Exporting Files

To use components and in this case `Header.js`, we must export them from their respective files and import them where needed.

```

1  const Header = () => {
2      return <h1>Welcome to My React App</h1>;
3  };
4
5  export default Header;
  
```



```
export default Footer;
```

*Tells we are
exporting the
component*

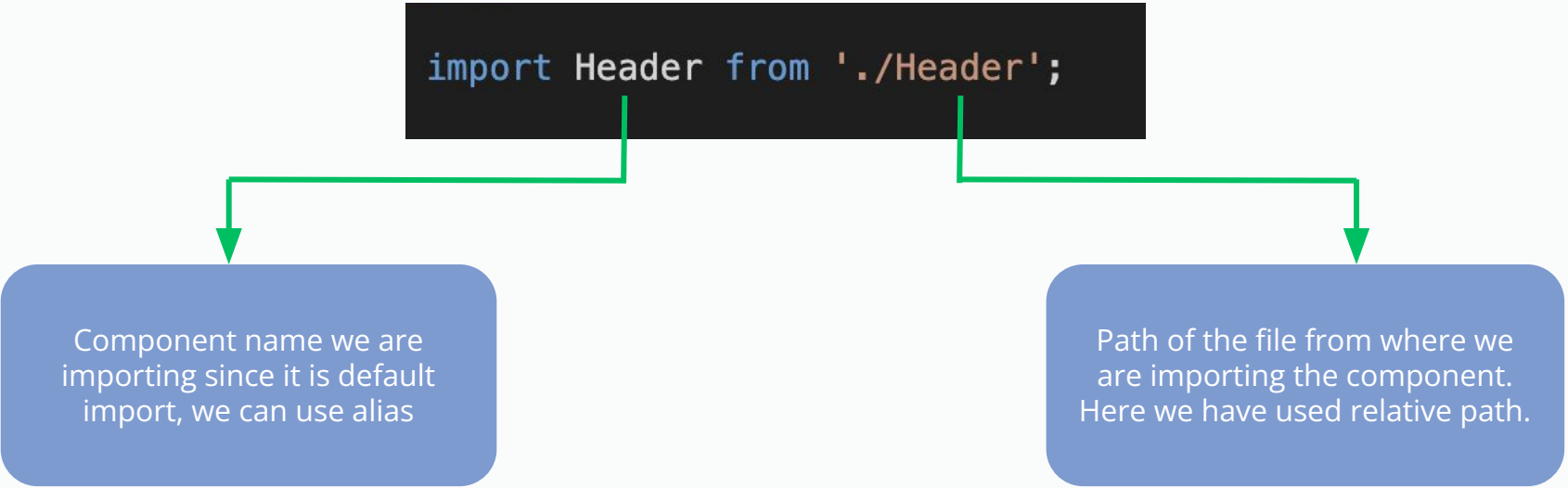
*And the export
type is default
export*

*We are
exporting
Footer
component*

Recap: Import Files

Let's dissect each part of our import statement:-

```
import Header from './Header';
```



Component name we are importing since it is default import, we can use alias

Path of the file from where we are importing the component. Here we have used relative path.

Recap: Passing props from parent to child

Prop passing from parent to child means sending data from a parent component to a child component in React using props (properties).

```

1  import Header from './Header';
2
3  const App = () => {
4    return (
5      <div>
6        <Header name="Narendra" />
7        <p>
8          This is the main
9          content of the app.
10        </p>
11        <Footer />
12      </div>
13    )
14  }
15
16  export default App;

```

Props are passed just like we write attributes in HTML elements

Props gets received in javascript variables as parameter to our component.

```

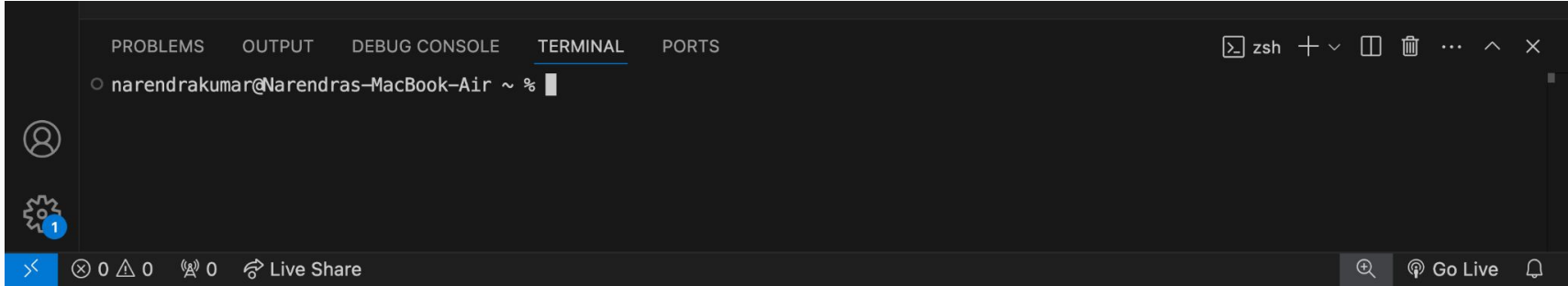
1  const Header = ({name}) => {
2
3    return (
4      <h1>Welcome, {name} to My React App</h1>
5    );
6  };
7
8  export default Header;

```

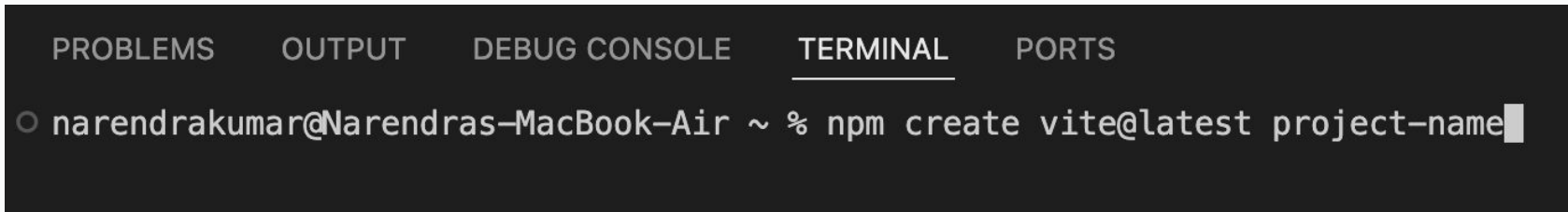
Using received prop value dynamically inside our JSX.

Recap: Creating a React App

Terminal would look like this:-



A screenshot of a VS Code terminal window. The terminal title bar shows 'zsh' and window control icons. The terminal content shows a shell prompt 'narendrakumar@Narendras-MacBook-Air ~ %' with a cursor. The VS Code interface includes tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (selected), and 'PORTS'. The bottom status bar shows icons for file explorer, search, and 'Go Live'.



A screenshot of a VS Code terminal window showing the command 'npm create vite@latest project-name' being entered at the shell prompt 'narendrakumar@Narendras-MacBook-Air ~ %'. The terminal tabs and status bar are the same as in the previous screenshot.

Run create vite@latest command with project directory name next to it

Recap: Installing and Launching App

And then you need to install all the packages by using “npm install” and run the app using “npm run dev”.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● narendrakumar@Narendras-MacBook-Air project-name % npm install

added 148 packages, and audited 149 packages in 8s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
  
```

Run “npm install” to install all the project dependencies packages

```

○ narendrakumar@Narendras-MacBook-Air project-name % npm run dev

> project-name@0.0.0 dev
> vite

VITE v6.2.0 ready in 307 ms

➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help
  
```

Run “npm run dev” to run your react application

Type this url in the address bar of your browser to preview your Project.

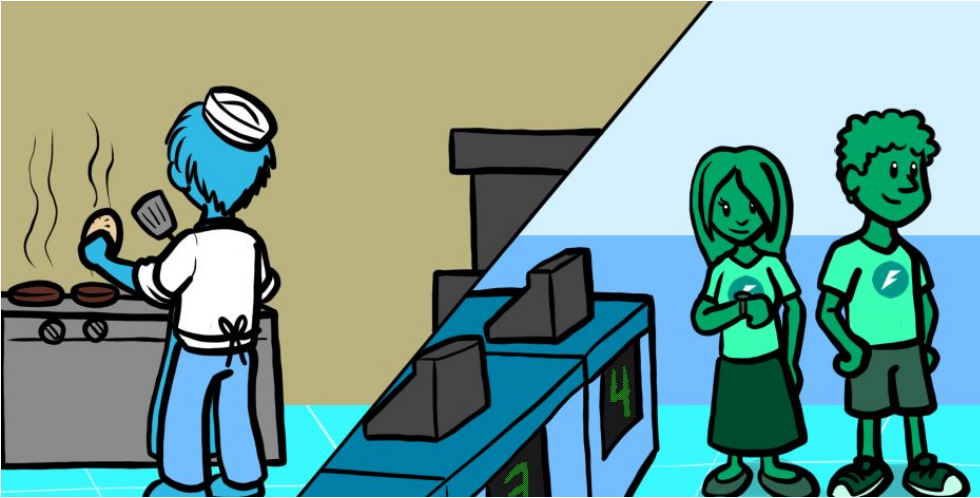


Introduction to `useEffect()`

Handling Side-Effects

Remember async operations in JS?

We used callbacks, promises, etc. But why? What was the issue with async operations?



Async operations do not execute immediately, so waiting for them blocks the execution of other tasks.

Use used promises to handle async ops

We used promises to handle async operations in JS. Here is an example:

```

1  function orderFood(order) {
2      return new Promise((resolve) => {
3          console.log(`Order placed: ${order}`);
4          setTimeout(() => {
5              resolve(`Order ready: ${order}`);
6          }, 3000); // Simulating food preparation time
7      });
8  }
9
10 console.log("Customer enters restaurant");
11
12 orderFood("Pizza").then((message) => {
13     console.log(message);
14 });
15
16 console.log("Other customers continue ordering...");

```

And then need to pass this orderFood() function to the event listener attached to an element.

Use use promises to handle async ops

We used promises to handle async operations in JS. Here is an example:-

```

1  // Mimicking useEffect with an event listener
2  document.getElementById("orderBtn").addEventListener("click", () => {
3      orderFood("Pizza").then((message) => {
4          console.log(message);
5      });
6  });
7
8  console.log("Other customers continue ordering...");
9
10
11 <html>
12     ....
13     <body>
14         <button id="orderBtn">
15             Order Pizza
16         </button>
17     </body>
18 </html>
  
```

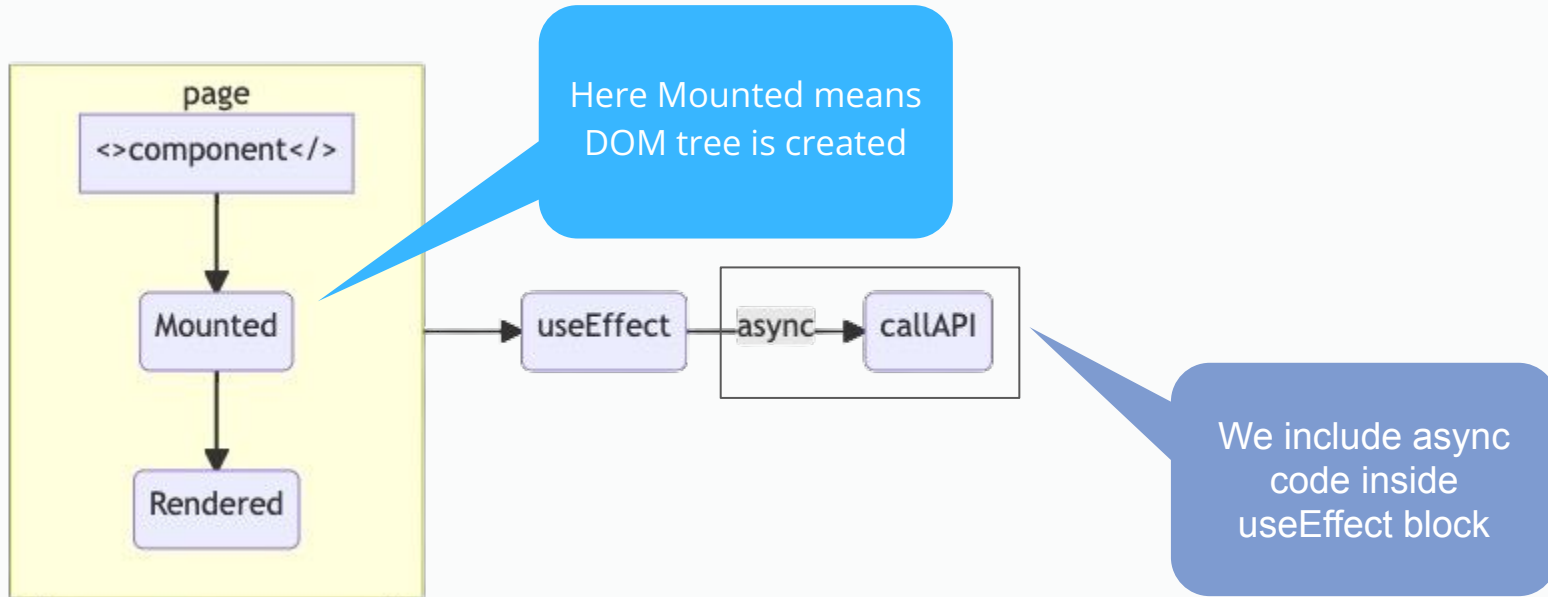


Do I need to do all this work to each element we want to listen?

Thankfully we have `useEffect()` hook in React.

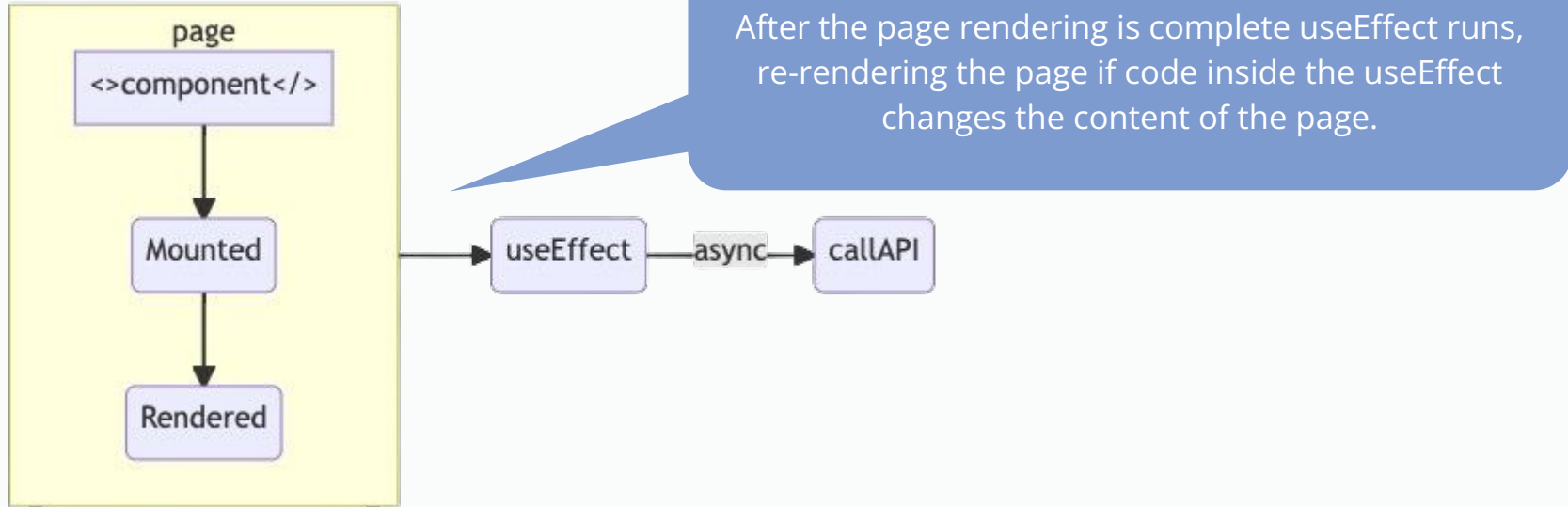
Don't worry we have `useEffect()` in React

`useEffect` in React handles side effects like data fetching, subscriptions, or DOM updates after render.



useEffect() simplifies the page updates

Just like promises facilitates the async operations to happen in right order, useEffect makes sure that async code runs only at the end allowing other parts of the page to load immediately.



Let's Understand with a Simple Example

Suppose you are provided a url from where you need to fetch the data and display its title in your web page.

```

1  document.addEventListener("DOMContentLoaded", () => {
2    const heading = document.createElement("h1");
3    heading.textContent = "Loading...";
4    document.body.appendChild(heading);
5
6    fetch("https://jsonplaceholder.typicode.com/todos/1")
7      .then(response => response.json())
8      .then(data => {
9        heading.textContent = data.title;
10       // Update content after fetching
11     });
12 });

```

If we use javascript we need to add event listener and create, insert dom elements by ourself

```

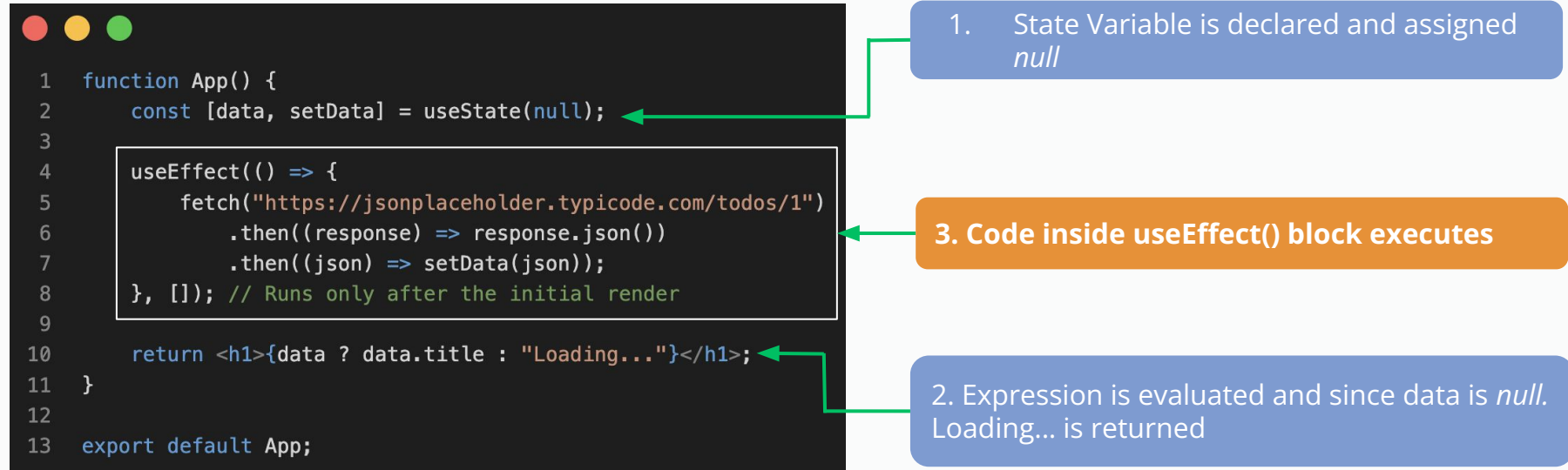
1  function App() {
2    const [data, setData] = useState(null);
3
4    useEffect(() => {
5      fetch("https://jsonplaceholder.typicode.com/todos/1")
6        .then((response) => response.json())
7        .then((json) => setData(json));
8    }, []); // Runs only after the initial render
9
10   return <h1>{data ? data.title : "Loading..."}</h1>;
11 }
12
13 export default App;

```

Whereas React runs useEffect at the end and if useEffect code changes content of the page, it reloads the latest copy of the page.

Order of execution...

Let's understand the order of execution of `useEffect()` with respect to rest of the code.



```

1  function App() {
2    const [data, setData] = useState(null);
3
4    useEffect(() => {
5      fetch("https://jsonplaceholder.typicode.com/todos/1")
6        .then((response) => response.json())
7        .then((json) => setData(json));
8    }, []); // Runs only after the initial render
9
10   return <h1>{data ? data.title : "Loading..."}</h1>;
11 }
12
13 export default App;
  
```

1. State Variable is declared and assigned *null*
2. Expression is evaluated and since data is *null*. Loading... is returned
3. Code inside `useEffect()` block executes

How React knows content has changed?

React tracks if its state variables are changed and when they are changed, a re-render of web page is triggered.

```

1  function App() {
2    const [data, setData] = useState(null);
3
4    useEffect(() => {
5      fetch("https://jsonplaceholder.typicode.com/todos/1")
6        .then((response) => response.json())
7        .then((json) => setData(json));
8    }, []); // Runs only after the initial render
9
10   return <h1>{data ? data.title : "Loading..."}</h1>;
11 }
12
13 export default App;

```

State Variable

Updating the state variable after fetch async operation is complete, making web page re-render with latest copy.

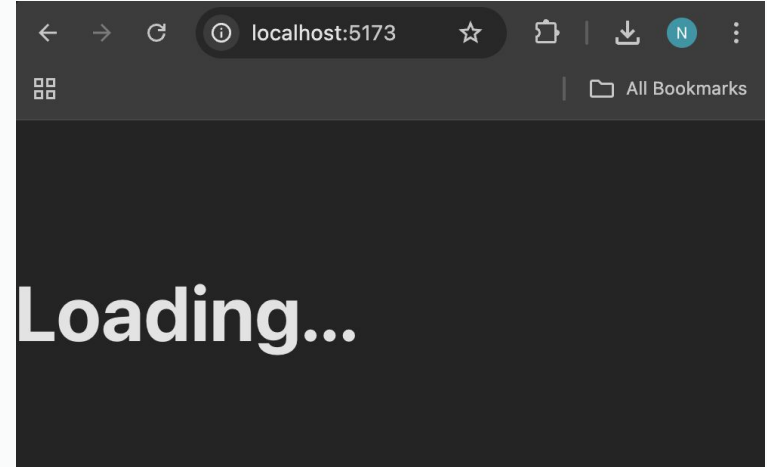
Initial Render: Data is not fetched yet

If your internet connection is fast enough then initial render might not be visible to you, in that case you need to slow down your internet connection from network tab of inspect tool.

```

1  function App() {
2    const [data, setData] = useState(null);
3
4    useEffect(() => {
5      fetch("https://jsonplaceholder.typicode.com/todos/1")
6        .then((response) => response.json())
7        .then((json) => setData(json));
8    }, []); // Runs only after the initial render
9
10   return <h1>{data ? data.title : "Loading..."}</h1>;
11 }
12
13 export default App;

```



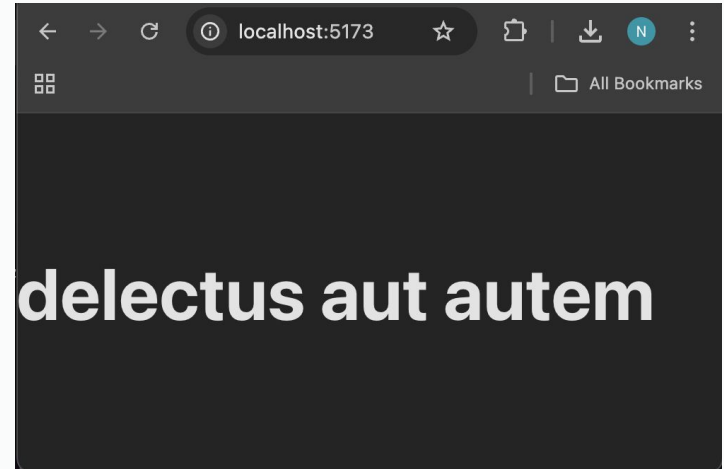
After fetch operation is completed

Have you observed that entire return block was executed once and as soon as state variable inside useEffect executed, JSX inside return statement got re-evaluated and re-rendered.

```

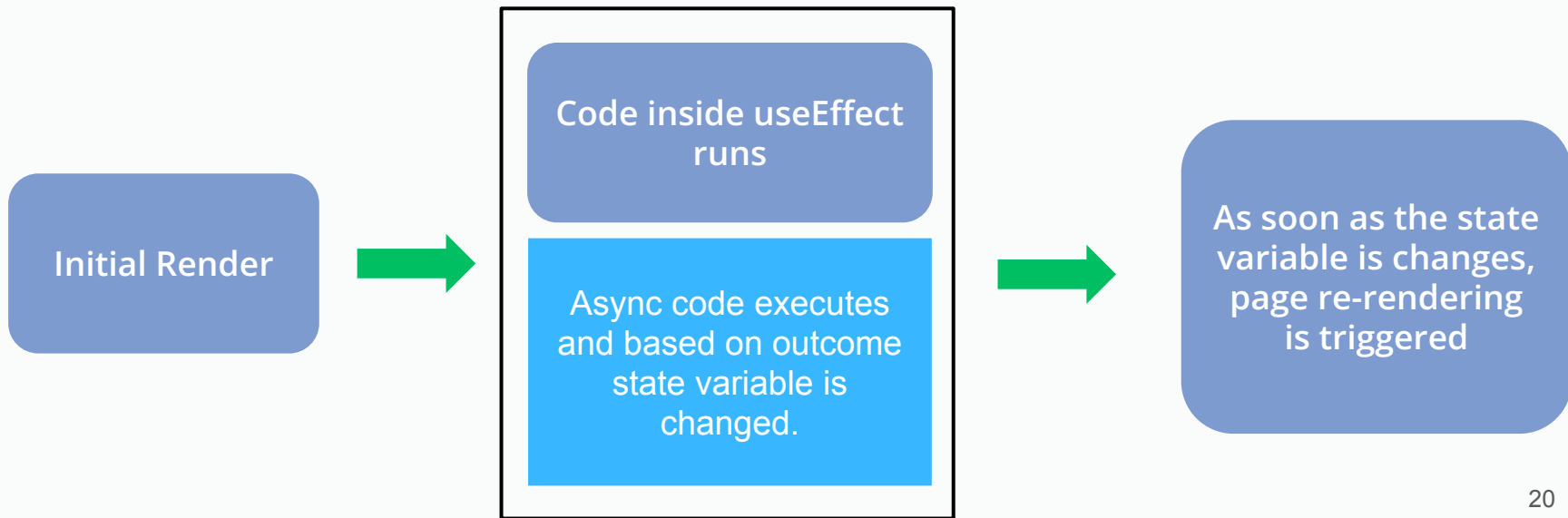
1  function App() {
2    const [data, setData] = useState(null);
3
4    useEffect(() => {
5      fetch("https://jsonplaceholder.typicode.com/todos/1")
6        .then((response) => response.json())
7        .then((json) => setData(json));
8    }, []); // Runs only after the initial render
9
10   return <h1>{data ? data.title : "Loading..."}</h1>;
11 }
12
13 export default App;

```



Key Takeaway

The only major work of `useEffect` is to execute its code after the initial render. We execute all async operations inside it, and make state changes if needed after async operations are complete, triggering the page re-render again.





Error Handling and Loading States

Managing failure states

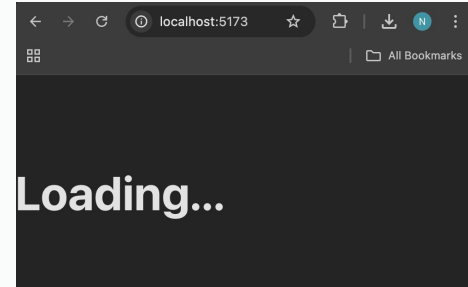
What if fetch api fails?

In case data fetching fails, then block is never executed and data state variable is never set. Would that mean, our app is always in Loading state? Isn't it inappropriate...

```

1  function App() {
2    const [data, setData] = useState(null);
3
4    useEffect(() => {
5      fetch("https://jsonplaceholder.typicode.com/todos/1")
6        .then((response) => response.json())
7        .then((json) => setData(json));
8    }, []); // Runs only after the initial render
9
10   return <h1>{data ? data.title : "Loading..."}</h1>;
11 }
12
13 export default App;

```



Always shows Loading state, but we should be showing that data fetch failed.

Using Error State to Manage Errors

We need to maintain an error state to handle failures. Check `response.ok` to detect API failure. If false, throw an error, which is caught in `catch` to update the error state.

```
import { useEffect, useState } from "react";

function App() {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/todos/1")
      .then((response) => {
        if (!response.ok) {
          throw new Error(`HTTP error! Status: ${response.status}`);
        }
        return response.json();
      })
      .then((json) => setData(json))
      .catch((err) => setError(err.message));
    // Catch and store error
  }, []); // Runs only after the initial render

  if (error) return <h1>Error: {error}</h1>;
  return <h1>{data ? data.title : "Loading..."}</h1>;
}

export default App;
```

If an error occurs, it is thrown and caught in `catch`, where the error state is updated with the message.

We check for an error first and display the error message if present; otherwise, we render the data.

Let's introduce a typo to check error message

Here we have intentionally introduced a typo to check if the error message is rendered in the screen.

```
import { useEffect, useState } from "react";

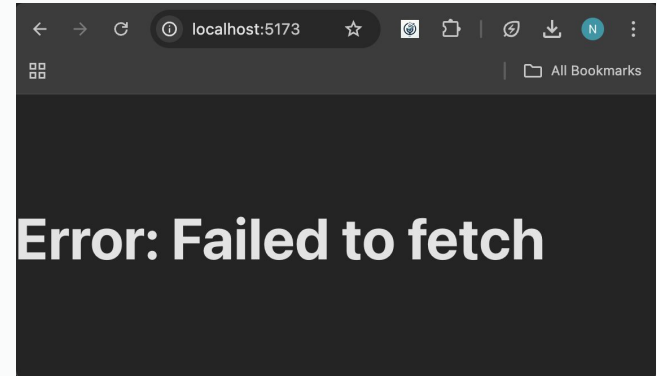
function App() {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);

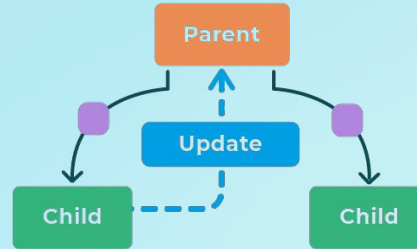
  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/todo/1")
      .then((response) => {
        if (!response.ok) {
          throw new Error(`HTTP error! Status: ${response.status}`);
        }
        return response.json();
      })
      .then((json) => setData(json))
      .catch((err) => setError(err.message));
  }, []); // Runs only after the initial render

  if (error) return <h1>Error: {error}</h1>;
  return <h1>{data ? data.title : "Loading..."}</h1>;
}

export default App;
```

Typo "todo" instead of "todos"





Lifting State Up

Child Talking to Parent

Recap: Create new components

Just like App.js which is the root component, we can build other components. For now we will build functional components.

We will build these components:-



Header

Footer

Creating Header.jsx

Here we have created Header component in the separate named Header.jsx:-

```
const Header = () => {  
  return <h1>Welcome to My React App</h1>;  
};
```

Header Component

Here we have created a simple header component with a simple message.

Pre-discussed: Including Header in App.js

We can simply import it in App.js file and use it here:-

```

1  // App.js
2  import React from "react";
3  import Header from "./Header";
4
5  const App = () => {
6    return (
7      <div>
8        <Header />
9        <p>This is the main content of the app.</p>
10     </div>
11   );
12 };
13
14 export default App;

```

While importing we have used relative path with respect to location of App.js

We can include our components using custom JSX tags, similar to HTML elements.

Recap: Creating Footer Component

Similarly we can create Footer component and use it in inside App like this:-

```
const Footer = () => {
  return (
    <p>
      @ 2025 My React App. All right reserved.
    </p>
  )
}
```

```
const App = () => {
  return (
    <div>
      <Header />
      <p>This is the main content of the app.</p>
      <Footer />
    </div>
  )
}

export default App;
```

*Awesome, I now can
create new
components...*



Recap: Customized Message in Header

Often, we need to display names or similar data in the webpage header. We can store the value and pass it to the **Header** component.

```

1  const Header = () => {
2    const name = "Narendra";
3
4    return (
5      <h1>Welcome, {name} to My React App</h1>
6    );
7  };
8
9  export default Header;

```



This doesn't seem right, we should be taking data from Home Page i.e. App.jsx

Recap: Passing props from parent to child

Instead we can use prop. Prop passing from parent to child means sending data from a parent component to a child component in React using props (properties).

```

1  import Header from './Header';
2
3  const App = () => {
4    return (
5      <div>
6        <Header name="Narendra" />
7        <p>
8          This is the main
9          content of the app.
10        </p>
11        <Footer />
12      </div>
13    )
14  }
15
16  export default App;

```

Props are passed just like we write attributes in HTML elements

Props gets received in javascript variables as parameter to our component.

```

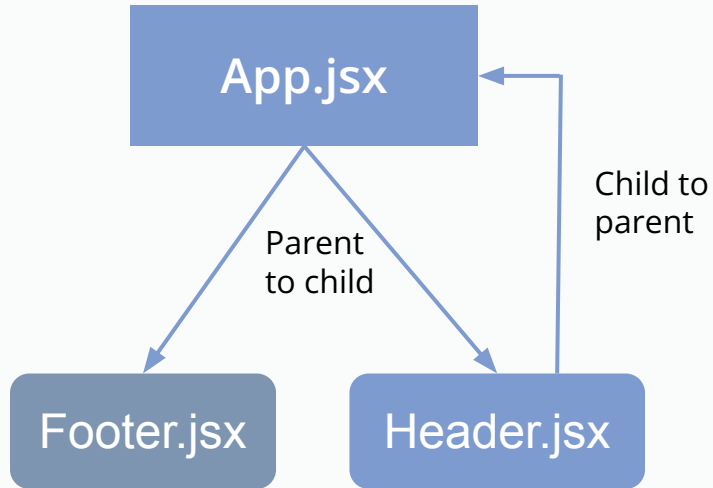
1  const Header = ({name}) => {
2
3    return (
4      <h1>Welcome, {name} to My React App</h1>
5    );
6  };
7
8  export default Header;

```

Using received prop value dynamically inside our JSX.

What if we need to pass prop child to parent

Lifting State Up moves state from a child to its parent, allowing shared management. The child updates the parent using callback functions passed as props.



Here parent App.jsx passes the state to its states to Footer.jsx and Header.jsx, Header.jsx modifies the state and send back to parent App.jsx

Lifting State Up: How it happens

The child updates the parent using callback functions passed as props.

```

1  import { useState } from 'react';
2  import Header from './Header';
3  import Footer from './Footer';
4
5  const App = () => {
6    const [name, setName] = useState("Narendra");
7
8    // Function to update the name (passed to child)
9    const handleNameChange = (event) => {
10      setName(event.target.value);
11    };
12
13    return (
14      <div>
15        <Header name={name} onNameChange={handleNameChange} />
16        <p>This is the main content of the app.</p>
17        <strong>Logged user: {name} </strong>
18        <Footer />
19      </div>
20    );
21  };
22
23  export default App;

```

*State update function
passed as a value in
onNameChange prop*

Lifting State Up: How it happens

Props passed are received in the child component, and then child can use it.

```
<Header name={name} onChange={handleChange} />
```

Received as prop

```
1  const Header = ({ name, onChange }) => {
2    return (
3      <div>
4        <h1>Welcome, {name}, to My React App</h1>
5        <input
6          type="text"
7          value={name}
8          onChange={onChange}
9          placeholder="Enter your name"
10       />
11      </div>
12    );
13  };
14
15  export default Header;
```

Here we have
attached received
function(onNameChange)
to onChange
event

Lifting State Up: How it happens

Let's understand the entire flow, what get passed to where:-

```

5  const App = () => {
6    const [name, setName] = useState("Narendra");
7
8    // Function to update the name (passed to child)
9    const handleNameChange = (event) => {
10      setName(event.target.value);
11    };
12
13    return (
14      <div>
15        <Header name={name} onChange={handleNameChange} />
16        <p>This is the main content of the app.</p>
17        <strong>Logged user: {name}</strong>
18        <Footer />
19      </div>
20    );

```

App.jsx

State
changer
function
passed as
prop

```

const Header = ({ name, onChange }) => {
  return (
    <div>
      <h1>Welcome, {name}, to My React App</h1>
      <input
        type="text"
        value={name}
        onChange={onChange}
        placeholder="Enter your name"
      />
    </div>
  );
};

```

Header.jsx

Prop
function
attached to
onChange
event

Prop got received

Lifting State Up: How it happens

Here we have attached state updater function to an even in the child component.

```
const handleNameChange = (event) => {
  setName(event.target.value);
};
```

App.jsx

handleNameChange()
got attached to
onChange event

```
const Header = ({ name, onNameChange }) => {
  return (
    <div>
      <h1>Welcome, {name}, to My React App</h1>
      <input
        type="text"
        value={name}
        onChange={onNameChange}
        placeholder="Enter your name"
      />
    </div>
  );
};
```

Header.jsx

Lifting State Up: How it happens

When user writes its name on input box, onChange event handler gets invoked and an event gets passed to its associated function i.e. onNameChange().

```
const Header = ({ name, onNameChange }) => {  
  return (  
    <div>  
      <h1>Welcome, {name}, to My React App</h1>  
      <input  
        type="text"  
        value={name}  
        onChange={onNameChange}  
        placeholder="Enter your name"  
      />  
    </div>  
  );  
};
```

Welcome, , to My React App

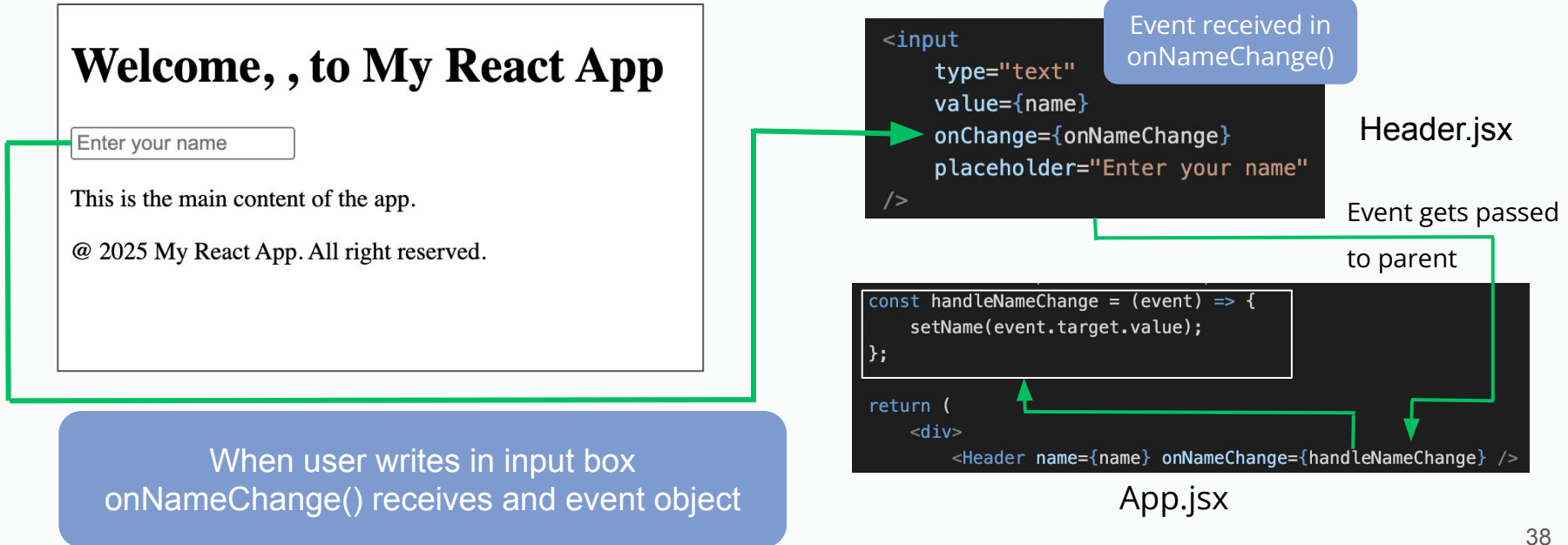
Enter your name

This is the main content of the app.

@ 2025 My React App. All right reserved.

Lifting State Up: How it happens

onNameChange() in turn corresponds to handleNameChange() function in the parent component.



Lifting State Up: How it happens

Hence we can update the state of parent function by passing state updater function in a prop.

```

4
5  const App = () => {
6    const [name, setName] = useState("Narendra");
7
8    // Function to update the name (passed to child)
9    const handleNameChange = (event) => {
10     setName(event.target.value);
11   };
12
13   return (
14     <div>
15       <Header name={name} onChange={handleNameChange} />
16       <p>This is the main content of the app.</p>
17       <strong>Logged user: {name}</strong>
18       <Footer />
19     </div>
20   );
21 };
22
23 export default App;

```

Updates the value
in react state
"name"

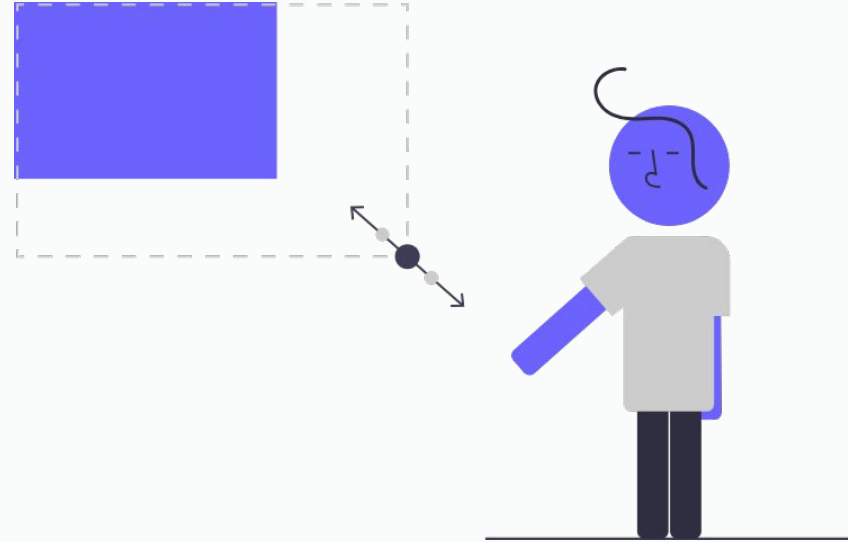
App.jsx

Resize Event

- Browser fires a **resize** event when window size changes
- Available globally on **window**

Examples of changes:

- Resize browser manually
- Rotate mobile device
- Open dev tools



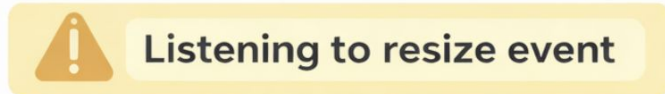
Basic Goal

- Track window width
- Update UI when window resizes

State needed:



Side effect:



Code Example

```
1  import { useState, useEffect } from "react";
2  function WindowSize() {
3    const [width, setWidth] = useState(window.innerWidth);
4    useEffect(() => {
5      const handleResize = () => {
6        setWidth(window.innerWidth);
7      };
8      window.addEventListener("resize", handleResize);
9      return () => {
10       window.removeEventListener("resize", handleResize);
11     }
12   }, []);
13   return <h1>Window width: {width}</h1>;
14 }
```

Why Cleanup is Required?

✗ Without Cleanup

- Memory leaks
- Multiple listeners
- Performance issues

✓ With Cleanup

- Listener removed on unmount
- Clean & safe component

```
1 //cleanup function
2 return () => {
3   window.removeEventListener("resize", handleResize);
4 };
```

In Class Questions

References

1. **MDN React Guide:** Comprehensive and beginner-friendly documentation for React
[https://developer.mozilla.org/en-US/docs/Learn/Tools and testing/Client-side JavaScript frameworks/React getting started](https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/React_getting_started)
2. **Websites and Tutorials:**
 - a.  w3schools: <https://www.w3schools.com/REACT/DEFAULT.ASP>
 - b.  codecademy: <https://www.codecademy.com/learn/react-101>
3. **Other Useful Resources**
 - a.  React GitHub Repository: <https://github.com/facebook/react>
 - b.  React Patterns & Best Practices: <https://reactpatterns.com/>

**Thanks
for
watching!**